

// Jivops

Docker Swarm

Guía completa — 30 lecciones

Fundamentos · Servicios y despliegue · Networking · Secretos y datos · Alta
disponibilidad · Producción avanzada

Contenido

Módulo 1: Fundamentos de Docker Swarm

- Lección 01 — Qué es Docker Swarm y qué problema resuelve
- Lección 02 — Arquitectura de Swarm — managers y workers
- Lección 03 — Inicializar un swarm y unir nodos
- Lección 04 — Servicios frente a contenedores individuales
- Lección 05 — Swarm frente a Kubernetes — cuándo elegir cada uno

Módulo 2: Servicios y despliegue

- Lección 06 — Servicios replicated y global
- Lección 07 — Rolling updates y rollback
- Lección 08 — Stacks y archivos Compose para Swarm
- Lección 09 — Restricciones de placement y preferencias
- Lección 10 — Health checks y auto-reparación

Módulo 3: Networking

- Lección 11 — Redes overlay en Swarm
- Lección 12 — Ingress routing mesh
- Lección 13 — Service discovery y DNS interno — VIP y dnsrr
- Lección 14 — Redes overlay cifradas
- Lección 15 — Publicar puertos — modo ingress frente a modo host

Módulo 4: Secretos, configs y datos

- Lección 16 — Docker secrets
- Lección 17 — Docker configs
- Lección 18 — Rotación de secretos
- Lección 19 — Datos persistentes y volúmenes en Swarm
- Lección 20 — Almacenamiento compartido y plugins de volúmenes

Módulo 5: Alta disponibilidad y operación

- Lección 21 — Consenso Raft y quórum de managers
- Lección 22 — Cuántos managers desplegar
- Lección 23 — Drenar nodos y mantenimiento
- Lección 24 — Recuperación tras pérdida de quórum
- Lección 25 — Backup y restauración del estado del swarm

Módulo 6: Producción avanzada

Lección 26 — Seguridad del swarm — mTLS, rotación de certificados y autolock

Lección 27 — Monitoreo de un swarm

Lección 28 — Logging centralizado

Lección 29 — CI/CD con Swarm

Lección 30 — Proyecto final: plataforma Docker Swarm lista para producción

Módulo 1: Fundamentos de Docker Swarm

Lección 01 — Qué es Docker Swarm y qué problema resuelve

Agrupar varios hosts Docker en un cluster, permite declarar cuántas réplicas de un servicio deben correr y se encarga de repartirlas entre nodos y de reemplazarlas si fallan.

Con el estado deseado declarado, Swarm compara continuamente lo que corre con lo que debería correr y corrige las diferencias; con docker run a mano nadie vigila que el contenedor siga vivo ni lo recrea en otro servidor si el host cae.

Lección 02 — Arquitectura de Swarm — managers y workers

Los managers mantienen el estado del cluster y deciden dónde va cada tarea; los workers ejecutan las tareas asignadas, y por defecto un manager también puede ejecutarlas.

Cada cambio de estado debe confirmarlo la mayoría de managers, así que más managers significa más coordinación y más latencia; los workers solo ejecutan tareas y se pueden añadir sin tocar el consenso.

Lección 03 — Inicializar un swarm y unir nodos

El host pasa a ser el primer manager, se generan los certificados y los tokens de unión, y se muestra el comando que otros nodos deben ejecutar para unirse.

El token decide el rol con el que entra el nodo; un manager puede administrar todo el cluster, así que su token es más sensible que el de un worker, que solo permite ejecutar tareas.

Lección 04 — Servicios frente a contenedores individuales

Es la definición declarativa de lo que debe correr (imagen, réplicas, red, límites); Swarm crea tareas, cada una con su contenedor, y las mantiene hasta que coincidan con esa definición.

Swarm no intenta arreglar la tarea que falló; descarta esa y crea otra nueva a partir de la definición del servicio, por eso la recuperación es rápida y no depende de diagnosticar el contenedor viejo.

Lección 05 — Swarm frente a Kubernetes — cuándo elegir cada uno

Un equipo pequeño que ya usa Docker y Compose, con necesidades de orquestación sencillas, que prioriza una curva de aprendizaje corta y poca infraestructura que operar.

Si el equipo ya escribe archivos Compose, puede reaprovecharlos con pocos cambios para desplegar en un cluster, sin aprender un formato de manifiestos completamente nuevo.

Módulo 2: Servicios y despliegue

Lección 06 — Servicios replicated y global

Replicated ejecuta el número de réplicas indicado repartidas por el cluster; global ejecuta una tarea en cada nodo disponible, útil para agentes de monitoreo o logging.

El agente debe estar físicamente en cada nodo para leer sus logs; el modo global lo garantiza y además arranca una tarea nueva cuando se añade un nodo, mientras que replicated podría dejar nodos sin agente.

Lección 07 — Rolling updates y rollback

Reemplaza las tareas por lotes según los parámetros parallelism y delay, de modo que siguen sirviendo réplicas antiguas mientras arrancan las nuevas, y puede revertir si la actualización falla.

Sin health check, una tarea cuenta como correcta apenas arranca el proceso aunque la aplicación esté rota; con él, la versión defectuosa se marca como no saludable y la actualización se detiene antes de reemplazar todas las réplicas buenas.

Lección 08 — Stacks y archivos Compose para Swarm

Describe en un único archivo versionable los servicios, redes, secretos y volúmenes de la aplicación y los despliega o actualiza con un comando, incluida la sección deploy propia de Swarm.

Swarm puede colocar la tarea en cualquier nodo y ese nodo necesita descargar la imagen por sí mismo; una imagen que solo existe en un equipo haría fallar las tareas programadas en los demás.

Lección 09 — Restricciones de placement y preferencias

Las constraints limitan los nodos elegibles según atributos o etiquetas (por ejemplo `node.role == worker` o una etiqueta de hardware); las preferences influyen en cómo se reparten las tareas entre los nodos elegibles.

La constraint es un requisito obligatorio (por ejemplo, un nodo con GPU); colocar la tarea donde no se cumple podría fallar o contradecir lo que pidió el administrador, así que Swarm la deja pending y el problema queda a la vista.

Lección 10 — Health checks y auto-reparación

Si el contenedor termina o su HEALTHCHECK falla repetidamente, marca la tarea como no saludable y crea una tarea nueva para volver al número de réplicas deseado.

El proceso puede seguir vivo pero bloqueado (por un deadlock o falta de recursos) sin atender conexiones; estar en ejecución solo indica que no ha terminado, mientras que el health check comprueba si la aplicación realmente responde.

Módulo 3: Networking

Lección 11 — Redes overlay en Swarm

Conecta contenedores que corren en distintos nodos como si estuvieran en la misma red local, encapsulando el tráfico (VXLAN) sobre la red física de los hosts.

El DNS interno de la red overlay resuelve el nombre del servicio hacia sus tareas actuales; aunque una tarea se mueva a otro nodo, el nombre sigue siendo el mismo y los clientes no tienen que enterarse.

Lección 12 — Ingress routing mesh

Todos los nodos del swarm escuchan en ese puerto y redirigen la petición hacia una tarea del servicio, aunque esa tarea no esté en el nodo que recibió la conexión.

El routing mesh reparte entre tareas una vez que la conexión llega a un nodo, pero no decide a qué nodo llegan los clientes; un balanceador externo da una dirección estable y evita enviar tráfico a un nodo caído.

Lección 13 — Service discovery y DNS interno — VIP y dnsrr

Por defecto devuelve una IP virtual (VIP) que balancea entre las tareas; con `endpoint_mode dnsrr` devuelve directamente las IP de las tareas y el reparto queda en manos del cliente.

Con `dnsrr` el cliente obtiene las IP de las tareas y, si las cachea, puede seguir usando una sola o una que ya no existe; con la VIP la dirección es estable y el reparto se hace dentro de Swarm en cada conexión.

Lección 14 — Redes overlay cifradas

Cifra el tráfico de datos entre contenedores de distintos nodos usando IPsec, mientras que el plano de control entre nodos ya viaja cifrado por defecto.

El tráfico de la overlay viaja por la red física entre hosts; en una red compartida o pública, alguien con acceso a ese segmento podría capturarlo, y el cifrado mantiene la confidencialidad aunque la red intermedia no sea de confianza.

Lección 15 — Publicar puertos — modo ingress frente a modo host

Ingress usa el routing mesh y abre el puerto en todos los nodos; host abre el puerto solo en el nodo donde corre la tarea, sin balanceo intermedio y conservando la IP de origen del cliente.

Con una tarea en cada nodo, el puerto queda disponible en todos sin pasar por el routing mesh; así se evita un salto extra y el proxy ve la IP real del cliente, útil para logs y reglas de acceso.

Módulo 4: Secretos, configs y datos

Lección 16 — Docker secrets

Se guarda cifrado en el log Raft de los managers y solo se entrega a los nodos que ejecutan tareas autorizadas, montándose como archivo en memoria dentro del contenedor.

Las variables de entorno se ven con `docker inspect` y pueden acabar en logs o procesos hijos; un archivo en memoria no se escribe en el disco del nodo y solo lo ve el contenedor autorizado.

Lección 17 — Docker configs

Para distribuir archivos de configuración no sensibles a las tareas sin incluirlos en la imagen, de modo que se puede cambiar la configuración sin reconstruir la imagen.

La imagen contiene el código y la config aporta los valores de cada entorno; así no hace falta reconstruir la imagen para cambiar un parámetro, y la imagen probada en pruebas es la misma que llega a producción.

Lección 18 — Rotación de secretos

Como los secretos son inmutables, se crea uno nuevo con otro nombre y se actualiza el servicio para quitar el antiguo y añadir el nuevo, lo que lanza un rolling update de las tareas.

Si se borrara, las tareas nuevas o reprogramadas no encontrarían el secreto que necesitan para arrancar; primero hay que actualizar el servicio para que deje de usarlo y después ya se puede eliminar.

Lección 19 — Datos persistentes y volúmenes en Swarm

Con el driver local el volumen vive en un único nodo, así que la tarea reprogramada no encuentra esos datos; hay que fijarla con constraints o usar almacenamiento compartido.

El volumen está en el disco del nodo que falló, y la tarea reprogramada en otro nodo arrancaría sin datos; hay que fijarla a un nodo o usar almacenamiento compartido o replicado.

Lección 20 — Almacenamiento compartido y plugins de volúmenes

Permiten que el mismo dato sea accesible desde el nodo donde se reprogramme la tarea, aunque añaden dependencia de ese sistema externo y de su disponibilidad.

Todos los nodos dependen del mismo sistema externo para acceder a los datos; si cae, las tareas con estado dejan de funcionar aunque el swarm esté sano, porque la alta disponibilidad de Swarm no cubre la del almacenamiento.

Módulo 5: Alta disponibilidad y operación

Lección 21 — Consenso Raft y quórum de managers

Es la mayoría de managers necesaria para confirmar cambios de estado; sin ella el swarm no puede aceptar cambios nuevos, aunque las tareas que ya estaban en marcha siguen ejecutándose.

Los contenedores los ejecuta cada nodo localmente y no dependen del manager para seguir vivos; lo que se pierde sin quórum es la capacidad de decidir cosas nuevas, como escalar o reprogramar tareas.

Lección 22 — Cuántos managers desplegar

Un número impar (3, 5 o 7): con N managers se tolera la pérdida de $(N-1)/2$, así que 3 toleran un fallo y 5 toleran dos, y con 4 o 6 no se gana tolerancia respecto al impar inferior.

Repartir los managers entre zonas evita que un fallo de una sola zona elimine la mayoría; pero cada cambio necesita confirmación de la mayoría, así que la latencia entre regiones muy lejanas ralentiza todo el consenso.

Lección 23 — Drenar nodos y mantenimiento

Swarm deja de asignarle tareas nuevas y reprograma las que tenía en otros nodos, permitiendo hacer mantenimiento sin interrumpir los servicios si el resto del cluster tiene capacidad.

Mover tareas que ya funcionan provocaría interrupciones innecesarias, así que Swarm solo usa el nodo recuperado para tareas nuevas o reprogramadas; si hace falta redistribuir, se fuerza actualizando el servicio.

Lección 24 — Recuperación tras pérdida de quórum

Se ejecuta `docker swarm init --force-new-cluster` en un manager superviviente para crear un cluster de un solo manager con el estado existente, y después se añaden managers nuevos.

Promover un nodo es un cambio de estado y debe confirmarlo la mayoría de managers; sin quórum no hay mayoría que lo acepte, por eso hace falta forzar un cluster nuevo desde un manager superviviente.

Lección 25 — Backup y restauración del estado del swarm

Se detiene Docker en un manager y se copia el directorio `/var/lib/docker/swarm` (estado Raft, certificados y claves); para restaurar se coloca en un nodo con Docker detenido y se inicia con `--force-new-cluster`.

El backup del swarm guarda el estado del orquestador (servicios, redes, secretos, configs), no los datos de los volúmenes; restaurarlo recrea la definición de los servicios pero no el contenido que manejaban.

Módulo 6: Producción avanzada

Lección 26 — Seguridad del swarm — mTLS, rotación de certificados y autolock

Cada nodo recibe un certificado de la CA interna del swarm y la comunicación entre nodos usa TLS mutuo; los certificados se rotan automáticamente (por defecto cada 90 días).

Autolock protege con una clave de desbloqueo (que no se guarda en el nodo) las claves que cifran el estado Raft; quien robe el disco no puede leer los secretos sin ella, y tras reiniciar un manager hay que introducirla para que arranque.

Lección 27 — Monitoreo de un swarm

El estado de los nodos, las réplicas en ejecución frente a las deseadas por servicio, el quórum de managers y los recursos de cada host, por ejemplo con cAdvisor y node-exporter junto a Prometheus.

Un servicio puede tener contenedores vivos y aun así menos de los necesarios para la carga; la diferencia entre réplicas deseadas y reales revela esos servicios degradados, algo que un simple recuento de contenedores no distingue.

Lección 28 — Logging centralizado

Las tareas pueden moverse entre nodos y los contenedores son efímeros, así que los logs deben enviarse mediante un driver de logging o un agente global a un sistema central, en lugar de depender de los archivos de cada nodo.

Los logs anteriores quedaron en el nodo antiguo y la tarea nueva escribe en otro, así que reconstruir la historia exigiría entrar nodo por nodo; un sistema central lo conserva todo en un lugar aunque el nodo ya no exista.

Lección 29 — CI/CD con Swarm

Construye la imagen, la publica en un registro con una etiqueta de versión inmutable y ejecuta docker stack deploy (o service update) contra un manager, apoyándose en el rolling update y el rollback de Swarm.

Latest puede apuntar a imágenes distintas en momentos distintos, y cada nodo podría descargar una versión diferente; no queda claro qué corría antes para volver a ello, mientras que una etiqueta de versión o un digest identifica exactamente el contenido desplegado.

Lección 30 — Proyecto final: plataforma Docker Swarm lista para producción

3 o 5 managers repartidos en zonas distintas con autolock, servicios con health checks y rolling update, secretos y configs fuera de la imagen, monitoreo y logging centralizados, y backups del estado del swarm y de los datos.

Intenta mencionar al menos dos pilares: managers impares con quórum, servicios con health checks y rolling update, secretos fuera de la imagen, monitoreo y logging centralizados o backups.